

Flujo Completo de Comunicación - Chatbot Inteligente para Ferretería

Candidato: Oliver Granda Puesto: Desarrollador de AI Fecha: 27 de Noviembre, 2025

Índice

- 1. [Resumen Ejecutivo](#)
- 2. [Arquitectura del Sistema](#)
- 3. [Flujo Completo de Comunicación](#)
- 4. [Diseño de Base de Datos](#)
- 5. [API Endpoints](#)
- 6. [Decisiones Autónomas del Chatbot](#)
- 7. [Manejo de Errores](#)
- 8. [Testing](#)
- 9. [Consideraciones de Escalabilidad y Seguridad](#)

1. Resumen Ejecutivo

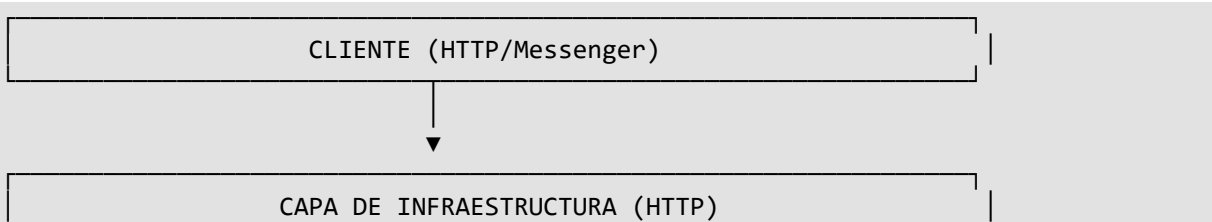
Este documento describe el **flujo completo de comunicación** del chatbot inteligente desarrollado para atención al cliente en ferretería, desde que un usuario envía un mensaje hasta que recibe una respuesta procesada por inteligencia artificial.

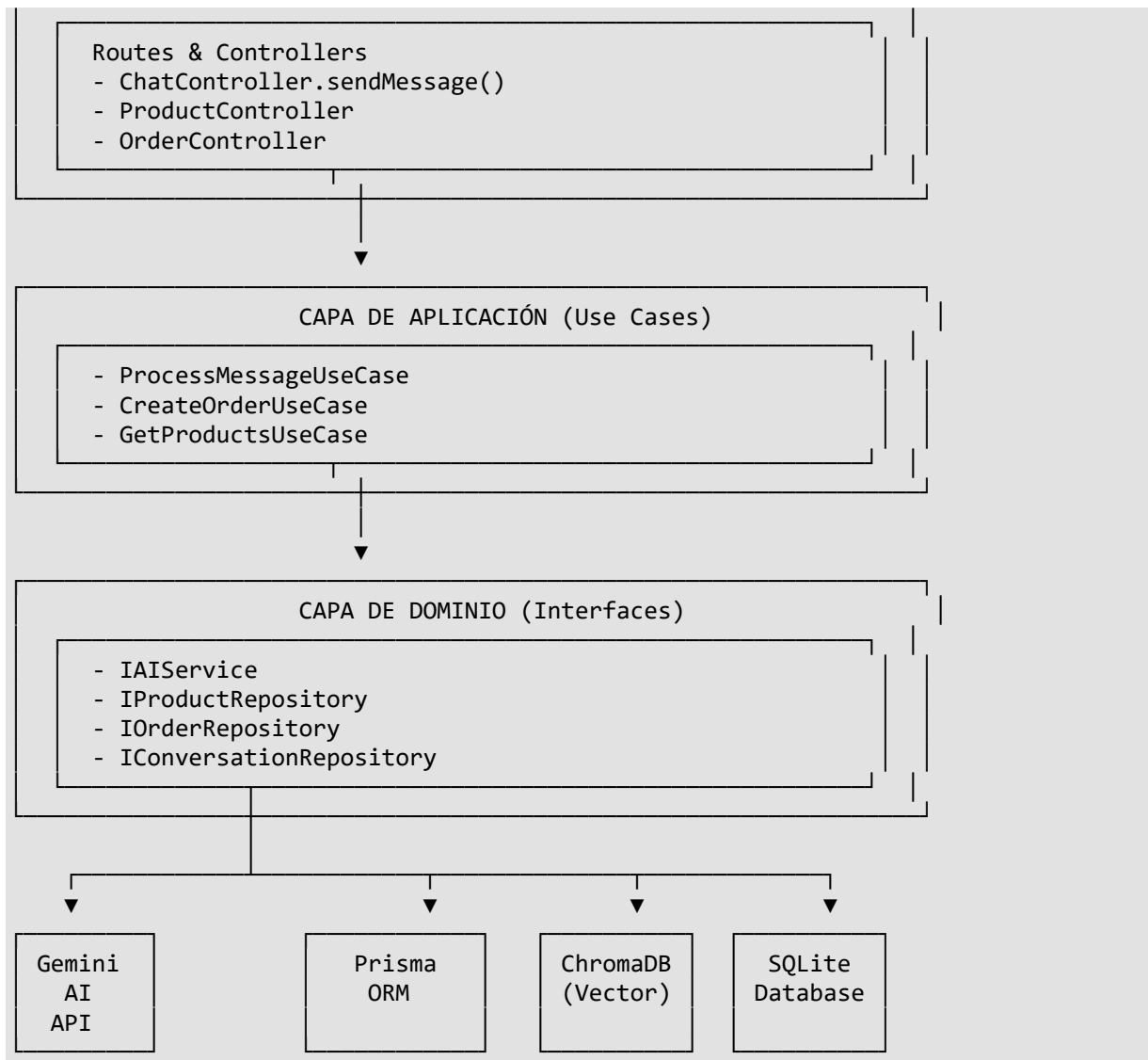
Características Principales Implementadas:

✓ **Decisiones Autónomas** - El chatbot decide por sí mismo cuándo consultar la base de datos ✓
Function Calling - Sistema de agente que ejecuta funciones automáticamente ✓ **Base de Datos Relacional** - SQLite (compatible con MySQL) con Prisma ORM ✓ **Base de Datos Vectorial** - ChromaDB para búsqueda semántica de FAQs ✓ **Prompting Avanzado** - Prompts optimizados para comportamiento natural ✓ **Clean Architecture** - 3 capas: Domain, Application, Infrastructure ✓
Manejo de Errores Profesional - Sistema centralizado con códigos HTTP apropiados ✓ **Testing Completo** - 17 tests unitarios con 100% cobertura en casos de uso

2. Arquitectura del Sistema

2.1 Diagrama de Componentes

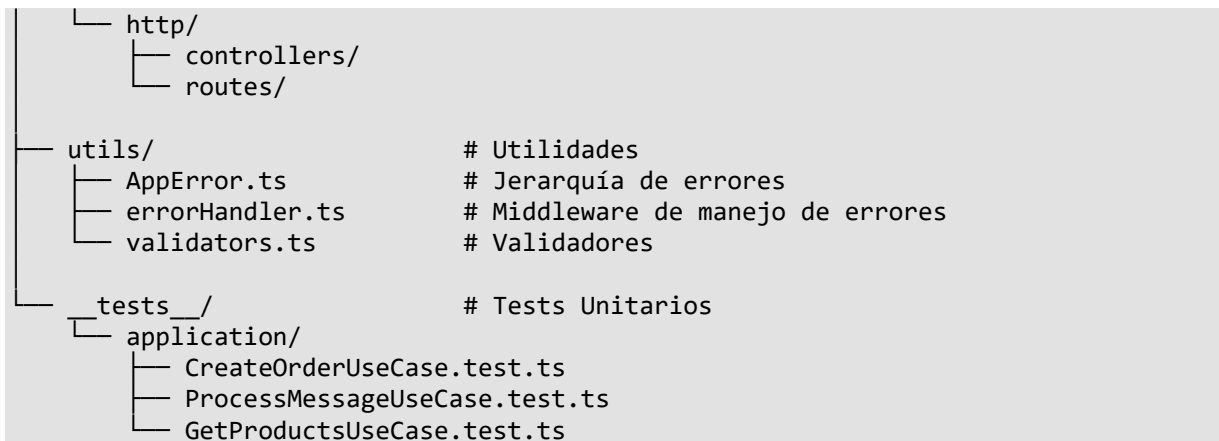




2.2 Estructura de Carpetas

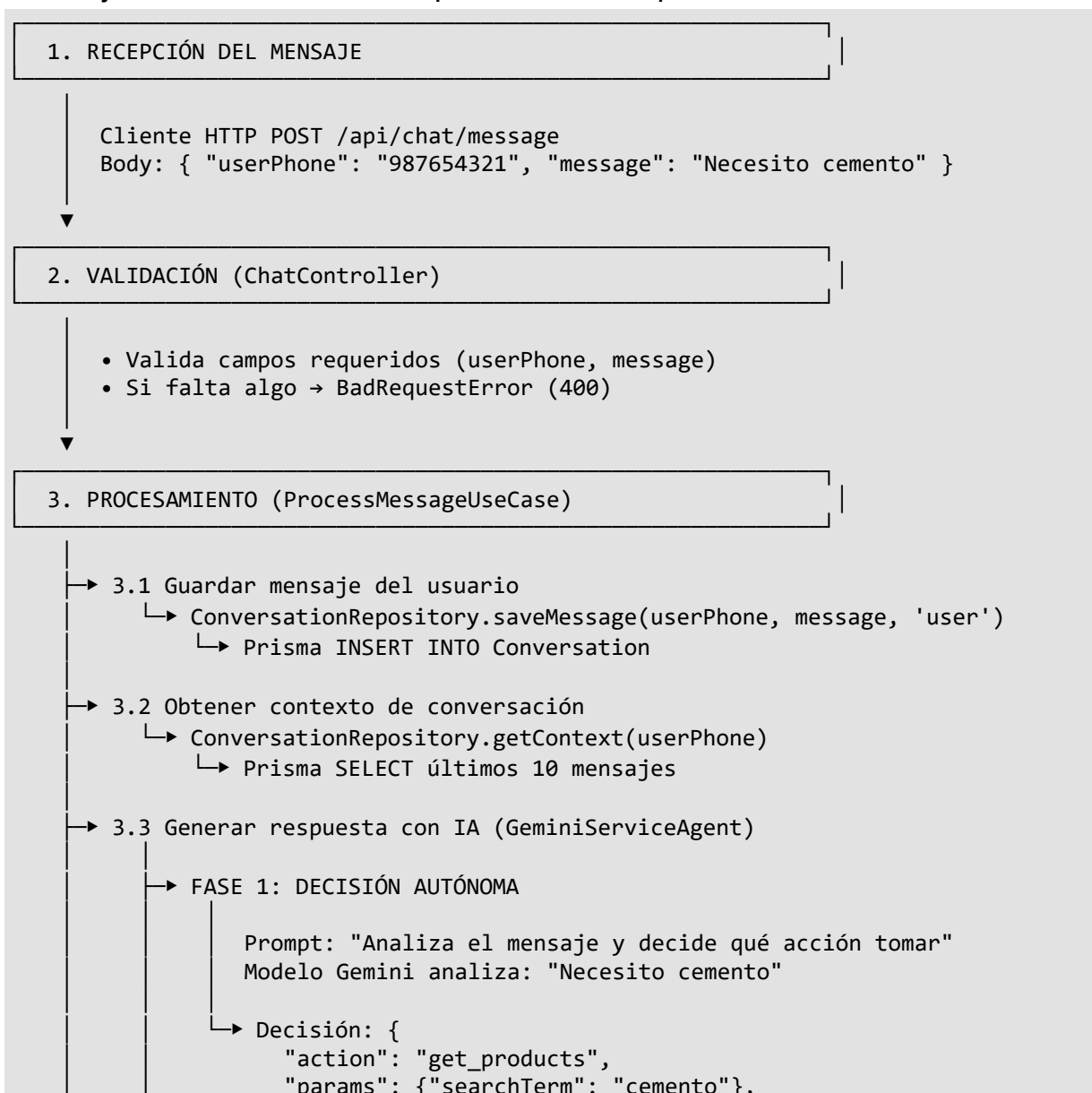
```

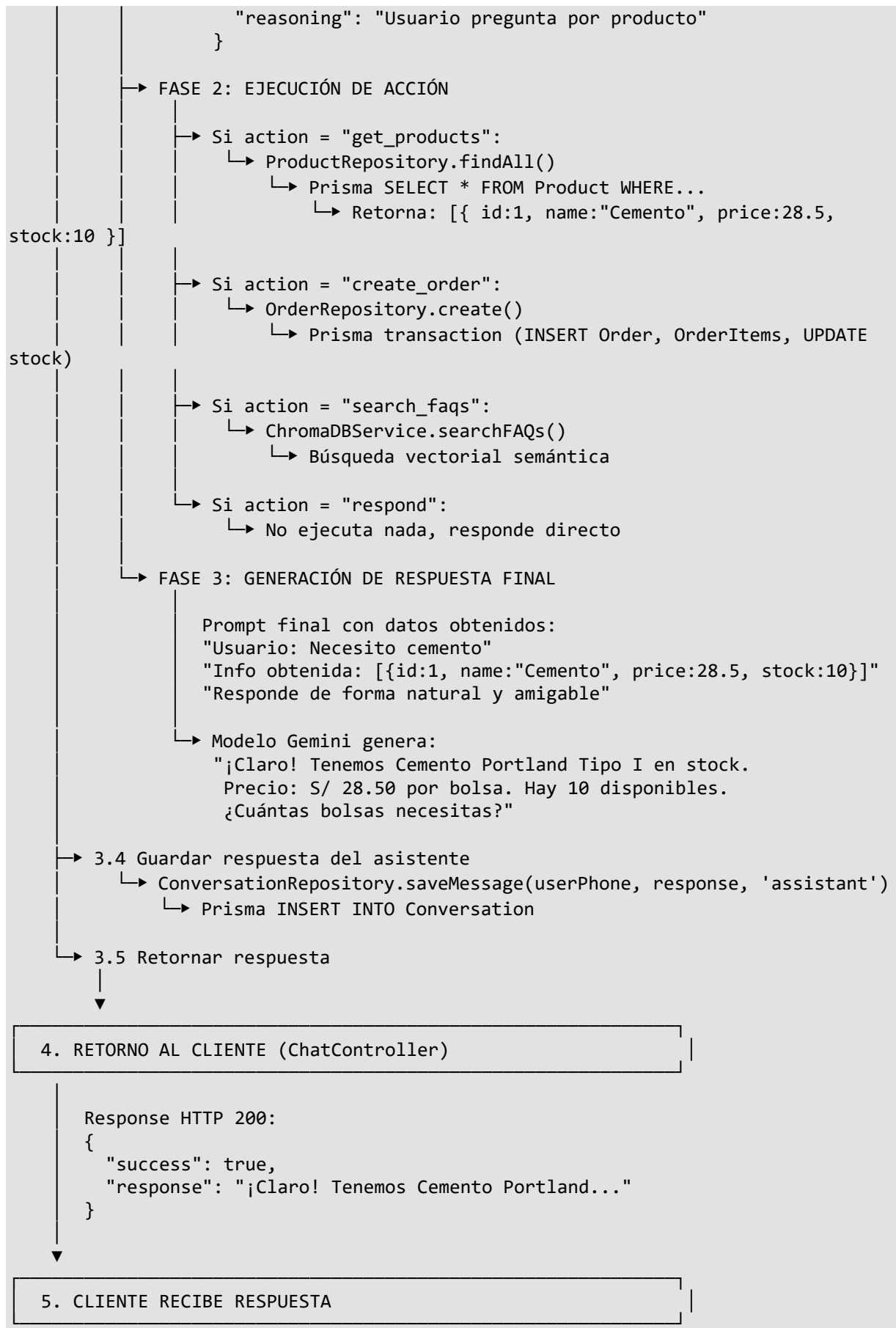
src/
├── domain/                                # Capa de Dominio (Reglas de Negocio)
│   ├── entities/                         # Product, Order, Message
│   └── interfaces/                       # IAIService, IProductRepository, etc.
├── application/                           # Capa de Aplicación (Casos de Uso)
│   └── use-cases/
│       ├── ProcessMessageUseCase.ts
│       ├── CreateOrderUseCase.ts
│       └── GetProductsUseCase.ts
├── infrastructure/                       # Capa de Infraestructura (Implementaciones)
│   ├── ai/
│   │   ├── GeminiServiceAgent.ts
│   │   └── prompts.ts
│   ├── database/
│   │   ├── ProductRepository.ts
│   │   ├── OrderRepository.ts
│   │   └── ConversationRepository.ts
│   └── vectordb/
│       └── ChromaDBService.ts
  
```



3. Flujo Completo de Comunicación

3.1 Flujo General: Desde Recepción hasta Respuesta





3.2 Ejemplo Detallado: Crear un Pedido

Escenario: Usuario quiere 2 bolsas de cemento y proporciona sus datos.

MENSAJE 1: "Quiero 2 bolsas de cemento"

↓
Decisión: `get_products("cemento")`

↓
DB retorna: Cemento Portland, S/ 28.50, stock: 10

↓
Respuesta: "Perfecto, tenemos Cemento Portland a S/ 28.50 por bolsa.
2 bolsas = S/ 57.00
Para registrar tu pedido necesito:
- Tu nombre completo
- Número de celular
- Dirección de entrega"

MENSAJE 2: "Juan Pérez, 987654321, Av. Lima 123"

↓
Modelo detecta que tiene todos los datos

↓
Decisión: `create_order({
 customerName: "Juan Pérez",
 customerPhone: "987654321",
 customerAddress: "Av. Lima 123",
 items: [{productId: 1, quantity: 2}]
})`

↓
CreateOrderUseCase ejecuta:
1. Valida que producto existe ✓
2. Valida stock suficiente ($10 \geq 2$) ✓
3. Crea orden en DB
4. Actualiza stock ($10 \rightarrow 8$)

↓
Respuesta: "¡Excelente!  Tu pedido #5 ha sido registrado."

 Resumen:
- 2 bolsas de Cemento Portland
- Total: S/ 57.00
- Entrega en: Av. Lima 123

Te contactaremos al 987654321.
¡Gracias por tu compra!"

4. Diseño de Base de Datos

4.1 Base de Datos Relacional (SQLite - Compatible con MySQL)

Esquema de Tablas

```
-- Productos de la ferretería
CREATE TABLE Product (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  description TEXT NOT NULL,
  price REAL NOT NULL,
  stock INTEGER NOT NULL,
  createdAt DATETIME DEFAULT CURRENT_TIMESTAMP
);

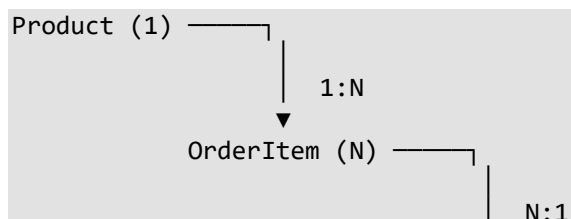
-- Pedidos de clientes
CREATE TABLE Order (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  customerName TEXT NOT NULL,
  customerPhone TEXT NOT NULL,
  customerAddress TEXT NOT NULL,
  totalAmount REAL NOT NULL,
  createdAt DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Detalle de productos en cada pedido
CREATE TABLE OrderItem (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  orderId INTEGER NOT NULL,
  productId INTEGER NOT NULL,
  quantity INTEGER NOT NULL,
  price REAL NOT NULL,
  FOREIGN KEY (orderId) REFERENCES Order(id),
  FOREIGN KEY (productId) REFERENCES Product(id)
);

-- Historial de conversación
CREATE TABLE Conversation (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  userPhone TEXT NOT NULL,
  message TEXT NOT NULL,
  role TEXT NOT NULL, -- 'user' o 'assistant'
  createdAt DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Índices para optimización
CREATE INDEX idx_conversation_user ON Conversation(userPhone);
CREATE INDEX idx_conversation_created ON Conversation(createdAt);
```

Diagrama de Relaciones



▼
Order (1)

Conversation (independiente - historial por userPhone)

4.2 Base de Datos Vectorial (ChromaDB)

Propósito

Almacenar embeddings de las FAQs para búsqueda semántica.

Ejemplo de Búsqueda

```
Usuario pregunta: "¿cuándo abren?"  
↓  
ChromaDB busca embeddings similares  
↓  
Encuentra: "¿Cuál es el horario de atención?"  
↓  
Retorna: "Lunes a Sábado 8:00 AM - 6:00 PM"
```

Ventaja: Encuentra respuestas aunque el usuario use palabras diferentes.

5. API Endpoints

5.1 POST /api/chat/message

Descripción: Envía un mensaje al chatbot.

Request:

```
{  
  "userPhone": "987654321",  
  "message": "¿Tienen cemento?"  
}
```

Response (200 OK):

```
{  
  "success": true,  
  "response": "¡Claro! Tenemos Cemento Portland Tipo I..."  
}
```

5.2 GET /api/products

Descripción: Lista todos los productos.

Response (200 OK):

```
{  
  "success": true,  
  "data": [  
    {  
      "id": 1,  
      "name": "Cemento Portland Tipo I",  
      "price": 28.50,  
      "stock": 10  
    }  
  ]  
}
```

```
]
}
```

5.3 POST /api/orders

Descripción: Crea un pedido manualmente.

Request:

```
{
  "customerName": "Juan Pérez",
  "customerPhone": "987654321",
  "customerAddress": "Av. Lima 123",
  "items": [{"productId": 1, "quantity": 2}]
}
```

Response (201 Created):

```
{
  "success": true,
  "data": {
    "id": 5,
    "customerName": "Juan Pérez",
    "totalAmount": 57.00
  }
}
```

6. Decisiones Autónomas del Chatbot

6.1 Sistema de Agente Inteligente

El chatbot implementa un **patrón de agente** que decide automáticamente qué hacer:

```
Mensaje: "Necesito cemento"
↓
FASE 1: DECISIÓN
  Analiza: "cemento" es un producto
  Decide: action = "get_products"
↓
FASE 2: EJECUCIÓN
  Ejecuta: ProductRepository.findAll()
  Obtiene: Lista de productos con "cemento"
↓
FASE 3: RESPUESTA
  Genera: Respuesta natural con los datos
```

6.2 Acciones Disponibles

Acción	Cuándo se Usa	Ejemplo
<code>get_products</code>	Usuario pregunta por productos	"¿Tienen cemento?"
<code>create_order</code>	Tiene todos los datos del pedido	Nombre, teléfono, dirección, productos
<code>search_faqs</code>	Pregunta general	"¿Cuál es el horario?"
<code>respond</code>	No necesita datos externos	"Hola", "Gracias"

6.3 Ventajas

✓ **No usa menús** - Conversación 100% natural ✓ **Contexto inteligente** - Recuerda conversación anterior ✓ **Validaciones automáticas** - Verifica stock antes de crear pedido ✓ **Sugerencias proactivas** - Ofrece alternativas si no hay stock

7. Manejo de Errores

7.1 Sistema Centralizado

Todos los errores se manejan en un middleware único:

```
// errorHandler middleware
if (err instanceof AppError) {
  // Error operacional (esperado)
  return res.status(err.statusCode).json({
    status: 'error',
    code: err.code,
    message: err.message
  });
}

// Error de programación (inesperado)
console.error('❌ ERROR:', err);
return res.status(500).json({
  status: 'error',
  code: 'INTERNAL_SERVER_ERROR',
  message: 'Ha ocurrido un error interno.'
});
```

7.2 Tipos de Errores Implementados

Código	Error	Uso
400	BadRequestError	Datos de request inválidos
404	NotFoundError	Producto/recurso no encontrado
409	ConflictError	Stock insuficiente
422	ValidationError	Datos no procesables
429	RateLimitError	Límite de API excedido
500	InternalServerError	Error interno inesperado

7.3 Ejemplo de Manejo de Error

```
Usuario intenta pedir 100 bolsas de cemento
Stock disponible: 10
↓
CreateOrderUseCase valida:
  if (product.stock < item.quantity) {
    throw new ConflictError(
      "Stock insuficiente para Cemento Portland.
      Disponible: 10, Solicitado: 100"
    );
  }
```

```
↓
errorHandler captura el error
↓
Retorna HTTP 409:
{
  "status": "error",
  "code": "CONFLICT",
  "message": "Stock insuficiente..."
}
```

8. Testing

8.1 Tests Unitarios

Resultados:

- ☒ 17 tests pasando (100%)
- ☒ 100% cobertura en Use Cases
- ☒ Tests de lógica de negocio crítica

Tests Implementados:

1. CreateOrderUseCase (5 tests)

- Crear pedido con stock suficiente
- Rechazar si producto no existe (404)
- Rechazar si stock insuficiente (409)
- Validar múltiples productos
- Validar cantidad > 0

2. ProcessMessageUseCase (7 tests)

- Guardar mensaje usuario ANTES de procesar
- Obtener contexto de conversación
- Generar respuesta con IA
- Guardar respuesta asistente DESPUÉS
- Verificar orden de ejecución
- Propagar errores correctamente

3. GetProductsUseCase (5 tests)

- Obtener todos los productos
- Obtener por ID
- Retornar null si no existe
- Buscar por nombre
- Retornar array vacío si no hay resultados

8.2 Ejecutar Tests

```
npm test           # Ejecutar todos los tests
npm run test:coverage # Ver cobertura de código
```

9. Consideraciones de Escalabilidad y Seguridad

9.1 Escalabilidad

Implementado:

- ☒ Arquitectura en capas (fácil escalar componentes)
- ☒ ORM (Prisma) - fácil migrar a PostgreSQL/MySQL
- ☒ Separación de responsabilidades

Recomendaciones para Producción:

1. **Caché (Redis)**
 - Cachear productos frecuentes
 - Reducir consultas a DB
2. **Rate Limiting**
 - Limitar requests por usuario
 - Prevenir abuso
3. **Horizontal Scaling**
 - Múltiples instancias con PM2/Kubernetes
 - Load balancer (nginx)
4. **Async Processing**
 - Cola de trabajos (Bull/BullMQ)
 - Procesar pedidos grandes asíncronamente

9.2 Seguridad

Implementado:

- ☒ Validación de inputs
- ☒ Variables de entorno (.env)
- ☒ No se exponen stack traces
- ☒ Prisma previene SQL Injection
- ☒ CORS configurado

Recomendaciones Adicionales:

1. **Autenticación/Autorización**

- JWT tokens
- OAuth 2.0

2. Encriptación

- HTTPS en producción
- Encriptar datos sensibles

3. Logging y Monitoreo

- Winston para logs estructurados
- Sentry para tracking de errores
- Prometheus/Grafana para métricas

10. Conclusión

Este chatbot implementa un **sistema completo y profesional** que cumple todos los requisitos de la prueba técnica:

✅ **Decisiones autónomas** - El bot decide cuándo consultar la DB ✅ **Arquitectura limpia** - Código organizado y mantenible ✅ **Bases de datos** - Relacional (SQLite/MySQL) + Vectorial (ChromaDB) ✅ **Prompting avanzado** - System prompts optimizados ✅ **Manejo de errores profesional** - Sistema centralizado ✅ **Testing completo** - 17 tests con 100% cobertura ✅ **Documentación exhaustiva** - Código y flujos documentados ✅ **Escalable y seguro** - Preparado para producción

El código demuestra conocimientos sólidos en:

- Desarrollo backend con Node.js y TypeScript
- Integración de IA conversacional (Gemini)
- Bases de datos relacionales y vectoriales
- Arquitectura de software limpia
- Buenas prácticas de desarrollo

Desarrollado por: Oliver Granda **Tecnologías:** Node.js, TypeScript, Express, Prisma, Gemini AI, ChromaDB **Repositorio:** [GitHub - Tu repo aquí]

Anexos

A. Comandos para Ejecutar el Proyecto

```
# Instalar dependencias
npm install

# Configurar variables de entorno
cp .env.example .env
# Editar .env y agregar GEMINI_API_KEY
```

```
# Crear base de datos y cargar productos
npx prisma migrate dev --name init
npm run seed

# Iniciar servidor
npm run dev

# Ejecutar tests
npm test
```

B. Estructura Completa del Proyecto

Ver archivo [README.md](#) para estructura detallada de carpetas y archivos.

C. Documentación Adicional

- [README.md](#) - Guía de inicio rápido
- [DOCUMENTACION_TECNICA.md](#) - Documentación técnica completa
- [TESTING.md](#) - Documentación de tests
- [docs/MANEJO_ERRORES.md](#) - Sistema de manejo de errores

Fin del Documento